

Edge cases (a template with the wrong variables)

— Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (checking a template's requirements before you ever call it). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the asymmetry](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the asymmetry

Basic Version

A template has blanks in it, like `{question}` . When you call `.invoke(...)` , you have to give it a dictionary with a value for every blank.

This exercise asks: what happens if you forget one? And what happens if you give it an extra one it didn't ask for?

Try both, on purpose, and read the actual error message (or lack of one) carefully — don't guess.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

A `ChatPromptTemplate` knows, from parsing its own template string, exactly which variable names it needs. When `.invoke(...)` is called, it checks the dictionary you passed against that known set of names *before* doing anything else — including before any network call.

This means a missing variable fails fast, locally, with a `KeyError`-style message naming the exact missing variable — not a vague downstream failure. An *extra* variable your dictionary provides but the template doesn't use behaves differently: the template only looks for the keys it needs; an extra key that isn't referenced in the template string is silently ignored, not an error. This asymmetry matters — a missing variable is loud and cheap to catch; an extra, unused one is silent, which means a typo'd variable name you *meant* to use, but that doesn't match the template, won't be caught this way — it'll just be silently dropped.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Both cases above are things you discover *by calling* `.invoke(...)` and seeing what happens. But a `ChatPromptTemplate` can actually tell you what it needs *before* you ever call it:

`prompt.input_variables` is a plain list of every variable name the template's own string references.

This turns "hope I remembered every key" into something you can check programmatically — the exact same "fail loudly, on purpose, before something bad happens" idea as Doc01's

`require_env()`, applied to a prompt's inputs instead of environment variables.

```
def validate_inputs(prompt, provided: dict) -> None:
    missing = set(prompt.input_variables) - set(provided.keys())
    if missing:
        raise ValueError(f"Missing prompt variable(s): {sorted(missing)}")
```

The real, harder question underneath this: `validate_inputs()` catches a *missing* key just like `.invoke()` already does on its own — so what does checking `input_variables` yourself actually buy you that you don't already get for free? Think about it before checking Hint 2: the answer is about *when* you find out, and what you can check about the *extra*, silently-ignored keys that `.invoke()` never tells you about at all.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both discover a template's requirements by calling `.invoke(...)` and observing what happens — one case errors, one doesn't. Advanced reads the template's own `input_variables` directly, so you can check what a template needs (and warn about keys you provided that it will silently ignore) without ever calling `.invoke(...)` at all — useful the moment you're validating many templates, or building a UI that needs to know a template's required fields ahead of time.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
build a template that needs one blank called question

test 1 - missing:
    call invoke with an empty dictionary
    catch the error, print it

test 2 - extra:
    call invoke with question filled in, plus an extra unused key
    print the result - does it error, or just work?
```

Here is almost the whole thing:

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template("Answer: {question}")

try:
    prompt.invoke({})
except Exception as e:
    print("Missing:", type(e).__name__, e)
```

Expected output: something naming `KeyError` and `'question'`. Add the extra-variable test call yourself before checking Hint 3.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
prompt = ChatPromptTemplate.from_template("Answer: {question}")

try:
    prompt.invoke({})
except KeyError as e:
    print(f"Missing variable -> KeyError: {e}")

result = prompt.invoke({"question": "What is LCEL?", "extra": "unused"})
print(f"Extra variable -> no error, result: {result}")
```

Notice the second call is not wrapped in `try/except` at all — because it's expected to succeed. If you find it doesn't, that's worth writing down as a real finding, not something to silently "fix" by assuming your first guess was right. Write both test calls yourself before checking Hint 3's Advanced level.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
function validate_inputs(prompt, provided) -> None:
    missing = prompt's required names, minus the ones actually provided
    if any missing: raise a clear error naming them

function warn_about_extra_inputs(prompt, provided) -> None:
    extra = the provided names, minus prompt's required names
    if any extra: print a warning naming them (not an error - just a heads-up)
```

Turning that into real code — fill in the missing piece yourself:

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template("Answer: {question}")

def validate_inputs(prompt: ChatPromptTemplate, provided: dict) -> None:
    missing = set(prompt.input_variables) - set(provided.keys())
    if missing:
        raise ValueError(f"Missing prompt variable(s): {sorted(missing)}")

def warn_about_extra_inputs(prompt: ChatPromptTemplate, provided: dict) -> None:
    # your turn: compute which keys in `provided` are NOT in
    # prompt.input_variables, and print a warning naming them if any
    # exist – these would be silently dropped by .invoke() with no
    # error at all, which is exactly the gap this function closes
    ...

good_inputs = {"question": "What is LCEL?", "extr": "typo'd key"}
validate_inputs(prompt, good_inputs)
warn_about_extra_inputs(prompt, good_inputs)
```

Expected output: `validate_inputs` passes silently (nothing's missing), but `warn_about_extra_inputs` should print something naming `extr` — catching the exact typo (`extr` instead of `extra`) that `.invoke()` alone would have silently ignored forever.

Fill in `warn_about_extra_inputs` yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both discover the missing/extra behavior by calling `.invoke(...)` once and reading what happens. Advanced writes 2 small, reusable functions around `prompt.input_variables` that check *both* directions before ever calling `.invoke(...)` — catching a missing required key the same way `.invoke()` already would, but also catching a typo'd extra key that `.invoke()` would otherwise swallow completely silently.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)